Assignment 3 – CO3: Constructors, Destructors and Operator Overloading

Name: T.Nikhitha
Register Number: 192424114
Subject: C++ Programming (Constructors, Destructors and Operator Overloading)
Assignment: 3 (CO3), 50 Marks, Individual
GitHub Repository: https://github.com/nikitha765432/T.Nikhitha-192424114-DSA0103-C-

Environment and Compilation

- Compiler: g++ 13.3.0 (Ubuntu), C++17
- Compile command: `g++ -std=c++17 -Wall -Wextra -g -fsanitize=address -o qN qN.cpp` (timing programs also built with `-O0` and `-O2`)
- Run: `./qN`. One folder per question (Q1 … Q7) holds the source, input files and output screenshots.
- All outputs, warnings and timings below come from actually compiling and running the code with this compiler.

Q1 – Constructors and Constructor Overloading (6 Marks)

Design

The `Book` class has one *target* constructor that validates the number of copies. All other constructors delegate to it, so the check exists in exactly one place.

```
#include <iostream>
#include <string>
class Book {
   std::string title;
   std::string author;
   int copies;
public:
   // target constructor: member initializer list + validation
   Book(const std::string& t, const std::string& a, int c)
      : title(t), author(a), copies(c < 0 ? 0 : c) {}
   // donated book: only a title (delegates)
   explicit Book(const std::string& t) : Book(t, "Unknown", 1) {}
   // empty record to fill later (delegates)
   Book() : Book("Untitled", "Unknown", 0) {}
   void print() const {
      std::cout << title << " | " << author << " | copies=" << copies << '\n';
   }
};
int main() {
   Book full("Algorithms", "CLRS", 5);
   Book donated("Gift Book");
   Book empty;
   Book bad("Broken", "X", -7);   // negative copies -> clamped to 0
```

```
    full.print(); donated.print(); empty.print(); bad.print();
}
```

No `Book` can hold an invalid count because every path ends in the target constructor, which clamps negative values to 0 (an alternative is to throw `std::invalid_argument`).

Analysis

- **When the compiler supplies a default constructor:** only when the class declares no constructor at all. Once any constructor is written (even a parameterised one), the compiler stops generating the default one. Write `Book() = default;` or define one yourself if it is needed.
- **Why initializer lists:** they initialise members directly. Assignment inside the body first default-constructs the member and then assigns, which is wasteful for `std::string`, and it is impossible for `const` members and references, which must be initialised in the list.
- **Initialisation order:** members are initialised in the order of declaration in the class, not the order written in the list. A mismatch is dangerous when one member's initial value depends on another, and `-Wreorder` reports it.
- **State of the objects after construction:**

Object	title	author	copies
full	"Algorithms"	"CLRS"	5
donated	"Gift Book"	"Unknown"	1
empty	"Untitled"	"Unknown"	0
bad (input -7)	"Broken"	"X"	0

Experimental Results

1. Every constructor, printed:

```
Algorithms | CLRS | copies=5
Gift Book | Unknown | copies=1
Untitled | Unknown | copies=0
Broken | X | copies=0
```

2. Default constructor removed, then `Book arr[3];`:

```
q1_nodef.cpp:4:24: error: no matching function for call to 'Book::Book()'
```

3. Wrong initialisation order (members declared `title, copies`, list written `copies(c), title(t)`), compiled with `-Wall -Wreorder`:

```
q1_reorder.cpp:2:37: warning: 'Book::copies' will be initialized after [-Wreorder]
q1_reorder.cpp:2:26: warning:   'std::string Book::title' [-Wreorder]
```

4. Negative copies: `Book("Broken", "X", -7)` prints `copies=0`, so the invalid value never reaches the object.

Design Question – explicit

Mark a constructor `explicit` when a single argument should not silently convert into the class. Without `explicit`, `void show(Book); show(std::string("Gift"));` compiles through implicit conversion. With `explicit Book(const std::string&)`, the same call fails:

```
q1_explicit.cpp:5:24: error: could not convert 'std::string("Gift")' from 'std::string' to 'Book'
```

The direct call `Book b("Gift");` still works.

Alternative: one constructor with default arguments

```
Book(std::string t = "", int copies = 0, std::string a = "Unknown")
   : title(t), author(a), copies(copies < 0 ? 0 : copies) {}
```

Time and space of object creation are O(1) in both designs. The default-argument version has a single constructor, so the copies check appears in one place and there is no delegation. The overloaded version states each situation explicitly and lets the donated case use different defaults, but it has more code paths. The default-argument version also acts as a default constructor and a converting constructor at once, which can allow unintended conversions.

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

Completed Code

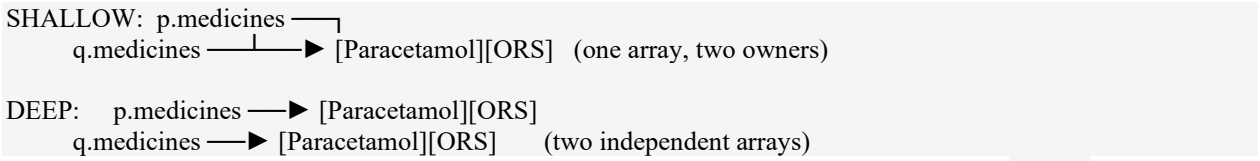
```
#include <iostream>
#include <string>
class Prescription {
   int n;
   std::string* medicines;
public:
   explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
   // TODO 1: deep-copy constructor
   Prescription(const Prescription& o) : n(o.n), medicines(new std::string[o.n]) {
      for (int i = 0; i < n; ++i) medicines[i] = o.medicines[i];
   }
   // TODO 2: copy assignment
   Prescription& operator=(const Prescription& o) {
      if (this == &o) return *this;          // self-assignment guard
      std::string* fresh = new std::string[o.n];
      for (int i = 0; i < o.n; ++i) fresh[i] = o.medicines[i];
      delete[] medicines;                    // release old array
      medicines = fresh;
      n = o.n;
      return *this;
   }
   ~Prescription() {
      delete[] medicines;                    // TODO 3
   }
   void set(int i, const std::string& m) { medicines[i] = m; }
   std::string get(int i) const { return medicines[i]; }
};
int main() {
   Prescription p(2);
   p.set(0, "Paracetamol"); p.set(1, "ORS");
   Prescription q = p;
   q.set(1, "Cetirizine");
   Prescription r(1);
   r = p;
   p.set(0, "Ibuprofen");
   std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
             << " | " << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Output (matches the expected output) and a clean AddressSanitizer run with no report:

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Analysis

Shallow copy (compiler-generated): `q.medicines` copies only the pointer, so p and q point to the same array. Changing q changes p, which is the reported bug.



Why the shallow version crashes: when p and q are destroyed, both destructors run `delete[]` on the same address. The second one is a double delete (undefined behaviour, usually an abort).

Rule of Three: if a class needs a user-written destructor, copy constructor or copy assignment, it almost certainly needs all three. They all deal with ownership of the same resource: the destructor releases it, the copy constructor and copy assignment must duplicate it so every owner releases its own copy.

Copy constructor is called when: (1) an object is initialised from another (`Prescription q = p;`), (2) an object is passed by value, (3) an object is returned by value (when not elided by the compiler).

Experimental Results

Sanitizer, version without the copy constructor (`-fsanitize=address`):

```
==125==ERROR: AddressSanitizer: heap-use-after-free on address 0x507000000090
READ of size 8 at 0x507000000090 thread T0
  #0 in Prescription::~Prescription() q2.cpp:21
  #1 in main q2.cpp:35
```

The first destructor freed the shared array; the second destructor then used freed memory. **Sanitizer, deep-copy version:** no error reported.

Copies made when passing:

Call	Copy-constructor messages
`void f(Prescription x)` (by value)	1
`void g(const Prescription& x)` (by const reference)	0

Self-assignment `p = p;` completes safely because of the `this == &o` guard (without it the array would be deleted before it is copied).

Time to copy one prescription (average of 20 copies, `-O2`):

Medicines	Time per copy
10	0.12 µs
1,000	8.2 µs
100,000	1,032 µs (about 1 ms)

Time grows roughly 100× for each 100× increase in size, which confirms O(n).

Conclusion

Rely on the compiler-generated copy constructor when every member copies correctly by itself (ints, `std::string`, `std::vector`, suitable smart pointers). Write your own when the class owns a raw resource (raw pointer, file handle, socket) where memberwise copy would share ownership.

Alternative: `std::vector<std::string>` (Rule of Zero)

```
class Prescription {
   std::vector<std::string> medicines;
public:
   explicit Prescription(int count) : medicines(count) {}
   // no destructor, copy constructor or copy assignment needed
};
```

Copying n medicines is O(n) time and O(n) space in both designs (each string is duplicated). A shallow copy is O(1) but unsafe because it shares ownership. The vector design costs the same as the hand-written deep copy with less code and no chance of a double delete.

Q3 – Dynamic Constructors and Destructors (7 Marks)

Design and Code

```
#include <iostream>
class Event {
   int rows, seats;
   char** map;          // array of row pointers
   int booked = 0;
public:
   Event(int r, int s) : rows(r), seats(s), map(new char*[r]) {
      for (int i = 0; i < rows; ++i) map[i] = new char[seats]();   // 0 = free
      std::cout << "Event created: " << rows << "x" << seats << '\n';
   }
   Event(const Event&) = delete;
   Event& operator=(const Event&) = delete;
   bool book(int r, int s)   { if (map[r][s]) return false; map[r][s] = 'X'; ++booked; return true; }
   bool cancel(int r, int s) { if (!map[r][s]) return false; map[r][s] = 0; --booked; return true; }
   ~Event() {
      for (int i = 0; i < rows; ++i) delete[] map[i];
      delete[] map;
      std::cout << "Event closed. Seats still booked: " << booked
                << " (" << rows << "x" << seats << ")\n";
   }
};
int main() {
   Event a(2, 3);               // stack
   a.book(0, 1);
   { Event b(1, 2); b.book(0, 0); }   // block scope
   Event* p = new Event(3, 3);    // heap
   p->book(2, 2); p->cancel(2, 2);
   delete p;                    // delete (not delete[]) for a single object
   Event z(0, 5);               // zero rows
}
```

Memory layout:



Analysis

- **Order:** local objects are constructed in order of declaration and destroyed in reverse order when `main` ends. A block-scope object is destroyed at the closing brace. A heap object is constructed by `new` and destroyed only when `delete` is executed.
- **Forgotten `delete`:** the destructor never runs, so memory leaks (and the closing summary is never printed). **`delete` instead of `delete[]`** on an array is undefined behaviour: only the first element's destructor may run and the allocator may be corrupted.
- **Destructor takes no arguments and cannot be overloaded:** it is called implicitly by the compiler when the lifetime ends, so there is no place to pass arguments, and therefore only one form can exist.

Experimental Results

Order of constructor and destructor messages (stack object `a`, block object `b`, heap object `*p`, stack object `z` with zero rows):

```
Event created: 2x3      <- a (stack)
Event created: 1x2      <- b (block)
Event closed. Seats still booked: 1 (1x2)  <- b destroyed at the closing brace
Event created: 3x3      <- *p (heap)
Event closed. Seats still booked: 0 (3x3)  <- delete p
Event created: 0x5      <- z
Event closed. Seats still booked: 0 (0x5)  <- z destroyed first (declared last)
Event closed. Seats still booked: 1 (2x3)  <- a destroyed last
```

Zero rows: `Event z(0, 5)` allocates a zero-length pointer array, the allocation loop does nothing, and the destructor still runs cleanly.

Sanitizer, `delete p;` removed:

```
==152==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 24 byte(s) in 1 object(s) allocated from: ...
SUMMARY: AddressSanitizer: 57 byte(s) leaked in 5 allocation(s).
```

The 5 allocations are the `Event` object (24 bytes), the row-pointer array (24 bytes) and three rows (3 bytes each). **With `delete p;` restored:** no report.

Memory for seat maps (one `char` per seat, one 8-byte pointer per row):

Seat map	Seat bytes	Row-pointer bytes	Allocations
10 × 20	200	80	11
100 × 200	20,000	800	101
1,000 × 2,000	2,000,000	8,000	1,001

Measured peak memory growth for the 1,000 × 2,000 map was 2,048 KB; the two smaller maps are too small to register in the process peak.

Conclusion

With RAII, acquiring memory in the constructor and releasing it in the destructor ties the resource's lifetime to the object's scope. Cleanup happens automatically on every exit path, including early returns and exceptions, so leaks and forgotten cleanups become far less likely.

Predict the Output

Step through `main`:

1. `Logger a("A")` prints `+A`
2. Block: `+B` then `-B`
3. `use(a)`: the parameter is copy-constructed with `n = "A" + "c" = "Ac"`, printing `copyAc`; the body prints `use`; the parameter is destroyed, printing `-Ac`
4. `new Logger("P")` prints `+P`, and `delete p` prints `-P`
5. `end` is printed, then `a` is destroyed at the end of `main`, printing `-A`

Predicted output:

```
+A +B -B copyAc use -Ac +P -P end -A
```

Actual output after compiling and running:

```
+A +B -B copyAc use -Ac +P -P end -A
```

The prediction and the actual output are identical, so there is no difference to explain. The copy exists because `use` takes its parameter by value. (The exact moment the parameter is destroyed, at the end of the call or the end of the full expression, is implementation-defined, but it is always after `use`.)

Alternative: contiguous block or vector of vectors

```
char* seats = new char[rows * cols]();        // index r * cols + c
// or: std::vector<std::vector<char>> map(rows, std::vector<char>(cols));
```

Access is O(1) in all designs. The row-pointer array needs rows + 1 allocations; the single block needs 1 and avoids the rows × 8 bytes of row pointers, and it is more cache-friendly. `vector<vector<char>>` also needs rows + 1 allocations plus 24 bytes of bookkeeping per row, but frees itself automatically.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

Completed Code

```
#include <iomanip>
#include <iostream>
class Money {
   long long paise;
public:
   Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
   // TODO 1
   Money operator+(const Money& o) const { Money r; r.paise = paise + o.paise; return r; }
   Money operator-(const Money& o) const { Money r; r.paise = paise - o.paise; return r; }
   Money& operator+=(const Money& o) { paise += o.paise; return *this; }
   bool operator<(const Money& o) const { return paise < o.paise; }
   // TODO 2
   bool operator==(const Money& o) const { return paise == o.paise; }
```

```
    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long a = m.paise < 0 ? -m.paise : m.paise;
        return os << (m.paise < 0 ? "-" : "") << "Rs " << a / 100 << '.'
                  << std::setw(2) << std::setfill('0') << a % 100;
    }
};
int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
    std::cout << Money(0, 75) + Money(0, 50) << '\n';   // carry test
    std::cout << Money(1) - Money(2) << '\n';           // negative result
}
```

Output (the first two lines match the expected output):

```
Rs 200.00
true
Rs 1.25
-Rs 1.00
```

Rs 0.75 + Rs 0.50 gives Rs 1.25: paise carry into rupees automatically because the value is stored as one integer number of paise.

Analysis

- **`+` returns a new object, `+=` returns `*this`:** `a + b` must not change a or b, so it builds a result. `+=` modifies the left operand and returns a reference so that chaining (`(a += b) += c`) works without a copy.
- **Members vs friends:** `+`, `-`, `+=`, `==`, `<` are members, since the left operand is a `Money` and they need access to private data. `<<` is a friend (non-member) because the left operand is `std::ostream`, a class we cannot modify. `>>` follows the same rule.
- **Why `<<` must be a non-member:** as a member it would have to be called on the left operand (`m << std::cout`), which is the wrong order.
- **Translation:** `a + b` becomes `a.operator+(b)`; `std::cout << c` becomes `operator<<(std::cout, c)`.

Experimental Results

- **Carry:** Rs 0.75 + Rs 0.50 printed `Rs 1.25`.
- **Negative result:** `Money(1) - Money(2)` stores -100 paise and prints `-Rs 1.00`. Decision: allow negative amounts, because they represent refunds and credits; a billing-only variant could reject them instead.
- **Readability:** `total = add(add(a, b), c); print(total);` becomes `std::cout << a + b + c;`, which reads like the arithmetic it performs.
- **Comparison with `double`** (adding Rs 0.10 one million times):

Version	Result	Error
`Money` (paise in `long long`)	Rs 100000.00	0
`double`	100000.0000013329	0.0000013329

The `double` version drifts because 0.10 cannot be stored exactly in binary; integer paise stays exact.

Engineering Decision

Money × Money has no meaning (rupee-squared). Sensible operators: `+`, `-`, `+=`, `-=`, comparisons, unary `-`, multiplication or division by a plain number (price × quantity, bill ÷ people), and money ÷ money giving a ratio. Not sensible: money × money, `%` with money, bitwise operators.

Alternative: non-member friend functions

`friend Money operator+(const Money& a, const Money& b) { Money r; r.paise = a.paise + b.paise; return r; }`
Time and space remain O(1) in every form. A non-member form allows `100 + m` as well as `m + 100`, because the implicit conversion `Money(100)` can apply to the left operand too. A member form allows only `m + 100`, because the left operand of a member operator is never converted. Named functions `add()` and `equals()` have the same cost but are less readable.

Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

Code

```
#include <iostream>
#include <string>
#include <vector>
class Time {
   int h, m, s;
public:
   Time(int hh = 0, int mm = 0, int ss = 0) : h(hh), m(mm), s(ss) {}
   Time& operator++() {                // prefix
      if (++s == 60) { s = 0; if (++m == 60) { m = 0; ++h; } }
      return *this;
   }
   Time operator++(int) { Time old = *this; ++(*this); return old; }   // postfix
   bool operator==(const Time& o) const { return h == o.h && m == o.m && s == o.s; }
   bool operator<(const Time& o) const {
      return h != o.h ? h < o.h : (m != o.m ? m < o.m : s < o.s);
   }
   friend std::ostream& operator<<(std::ostream& os, const Time& t) {
      return os << t.h << ':' << t.m << ':' << t.s;
   }
};
class Playlist {
   std::vector<std::string> tracks;
public:
   void add(const std::string& t) { tracks.push_back(t); }
   std::string& operator[](int i) {
      if (i < 0 || i >= (int)tracks.size()) { std::cerr << "Index out of range\n"; static std::string d; return d; }
      return tracks[i];
   }
   const std::string& operator[](int i) const {
      if (i < 0 || i >= (int)tracks.size()) { std::cerr << "Index out of range\n"; static const std::string d; return d; }
      return tracks[i];
   }
};
```

Analysis

- **Prefix vs postfix:** the compiler calls `operator++(int)` for postfix. The unused `int` is a dummy parameter whose only role is to give a different signature.
- **Postfix returns a copy:** it must return the *old* value after incrementing, so it saves a copy, which costs an extra construction and destruction. Prefix increments in place and returns a reference.
- **Const and non-const `operator[]`:** the non-const version returns a modifiable reference for `playlist[i] = "x"`; the const version is needed for `const Playlist&` objects and returns a const reference.
- **Out of range:** report an error (print, assert or throw `std::out_of_range`) and never read memory outside the container.

Experimental Results

`a = t++` and `b = ++t` starting from `Time t(0,0,0)`:

```
a=0:0:0 b=0:0:2 t=0:0:2
```

Roll-over: `Time(0,59,59)` incremented with prefix `++` prints `1:0:0`; `Time(0,59,58)` incremented twice with postfix `++` also prints `1:0:0`. **Comparison:** `Time(0,0,2) < Time(0,1,0)` is `true`, and `Time(1,2,3) == Time(1,2,3)` is `true`.

Playlist range check: `playlist[0]` and `playlist[1]` return `Song A` and `Song B`. `playlist[-1]` and `playlist[size]` each print `Index out of range` and return a harmless empty string.

10,000,000 increments:

Build	Prefix	Postfix
`-O0`	21.4 ms	48.5 ms
`-O2`	5.0 ms	4.5 ms

At `-O0` postfix is about 2.3× slower because of the extra copy; at `-O2` the optimiser removes the copy and the difference disappears (the small gap is measurement noise).

Conclusion

For class types, `++i` avoids creating a temporary copy. For ints the compiler optimises the difference away, but for larger objects (iterators, big classes) prefix is never slower, so it is the safe habit.

Predict the Output

`Counter c{5};` → `a = c++` stores the old value 5 in a (c becomes 6) → `b = ++c` increments to 7 and copies 7 into b.

Predicted output: `5 7 7`

Actual output after compiling and running:

```
5 7 7
```

The prediction matches. a and b differ because postfix returns the value from before the increment, while prefix returns the value after. The postfix version makes the copy (`Counter old = *this`).

Alternative: `at(i)` or unchecked `[]`

`at(i)` throws `std::out_of_range`, which forces the caller to handle the error; unchecked `[]` is fastest but undefined on bad indexes. All three are O(1) per access; the checked version adds one comparison. Measured cost for 10,000,000 accesses:

Build	Checked []	Unchecked
-O0	68.6 ms	44.6 ms
-O2	17.4 ms	10.9 ms

The range check adds roughly 25 ms at -O0 and 6 ms at -O2 per 10 million accesses, which is small compared with the safety it provides.

Q6 – The Rules of Operator Overloading (7 Marks)

Code

```
#include <cstring>
#include <iostream>
class Text {
   char* data;
   size_t len;
public:
   Text(const char* s = "") : data(new char[std::strlen(s) + 1]), len(std::strlen(s)) {
      std::strcpy(data, s);
   }
   Text(const Text& o) : data(new char[o.len + 1]), len(o.len) { std::strcpy(data, o.data); }
   Text& operator=(const Text& o) {
      if (this == &o) return *this;
      char* fresh = new char[o.len + 1];
      std::strcpy(fresh, o.data);
      delete[] data;
      data = fresh; len = o.len;
      return *this;
   }
   ~Text() { delete[] data; }
   Text& operator+=(const Text& o) {
      char* fresh = new char[len + o.len + 1];
      std::strcpy(fresh, data); std::strcat(fresh, o.data);
      delete[] data; data = fresh; len += o.len;
      return *this;
   }
   friend Text operator+(Text a, const Text& b) { a += b; return a; }
   bool operator==(const Text& o) const { return std::strcmp(data, o.data) == 0; }
   bool operator!=(const Text& o) const { return !(*this == o); }
   friend std::ostream& operator<<(std::ostream& os, const Text& t) { return os << t.data; }
};
```

Members are declared in the order `data`, `len`, and the initializer lists follow that order, so `-Wreorder` stays silent.

Analysis

- **Operators that cannot be overloaded:** :: (scope resolution – operates on names, not objects), . (member access – overloading it would make normal member access impossible), .* (pointer-to-member access), ?: (ternary, has three operands and short-circuit evaluation that a function call cannot replicate), sizeof (evaluated at compile time on types).
- **Rules:** precedence, associativity and arity stay as for built-in operators; new operator symbols cannot be invented; at least one operand must be a user-defined type (so int + int can never change).
- **operator= must be a member:** the language requires it, so that assignment always has a modifiable left operand of the class and the compiler can supply a default one consistently.
- **When to use a friend:** when the left operand is not our class (<<, >>) or when symmetric conversions on both operands are wanted (operator+), and the function needs private access.

Experimental Results

Output of the test program (compiled with -fsanitize=address; the sanitizer reported no errors):

```
Hello, SMS Gateway  | copies made by a+b+c: 3
empty: [] size 0
self-assign: self
long msg size 2000
true true
```

- **Copies for a + b + c: 3.** The first + copies a into its by-value parameter (1) and returns that parameter by copying it, since Text has no move constructor (2). The second + receives the temporary directly (no copy in C++17) and again copies on return (3).
- **Sanitizer:** the code with the + 1 for the terminating '\0' and delete[] in the destructor and assignment is clean. Typical bugs it catches here are a missing + 1 (heap-buffer-overflow) and a missing delete[] (leak).
- **Self-assignment** s = s; leaves self unchanged; **empty message** has length 0; a **long message** (1,000 + 1,000 characters) gives length 2,000.
- **100,000 appends of "ab":**

Version	Time
Text (reallocates on every +=)	0.9 s at -O0; up to 4.9 s in another -O2 run (allocator-dependent)
std::string	about 0.6–1.1 ms

Text copies the whole string on every append, so n appends cost $O(n^2)$ characters copied; std::string grows its capacity geometrically, so appends are amortised O(1).

Conclusion

Writing Text teaches ownership, the Rule of Three, deep copying, why a naive += is quadratic, and how much std::string does for us (small-string optimisation, capacity growth). Writing such a class is still reasonable for learning, for constrained or embedded environments, or when a special memory layout is needed.

Alternative: `std::string`, or `+` via `+=`

With `operator+(Text a, const Text& b) { a += b; return a; }` (the version used above), evaluating `a + b + c` copies about n1 + n2 characters for the first `+` and n1 + n2 + n3 for the second, with a temporary of size n1 + n2 held at the same time. With `std::string` the characters copied are the same order, O(n1 + n2 + n3), but capacity growth avoids repeated reallocation, and the temporary space is O(n1 + n2).

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Stage 1 – Interpret the Problem (2 Marks)

Data: real part and imaginary part (two `doubles`). **Operations:** add (series), multiply and divide (parallel), subtract, equality, printing in rectangular form, magnitude, and safe division.

Constructors: default (0 + 0j); from real part only (a resistor: R + 0j); from real and imaginary parts (inductor 0 + jωL, capacitor 0 − j/(ωC)).

Measurable requirements:

1. `Complex(3)` gives 3 + 0j.
2. (1+2j) + (3+4j) = 4+6j.
3. |3+4j| = 5.
4. Z1 ‖ Z2 = (Z1 × Z2) / (Z1 + Z2) matches a hand calculation to 6 decimal places.
5. Dividing by 0 + 0j reports an error and never produces NaN or a crash.

Stage 2 – Design (3 Marks)

Operator	Formula (z1 = a + bj, z2 = c + dj)	Form
`+`	(a+c) + (b+d)j	member
`-`	(a−c) + (b−d)j	member
`*`	(ac − bd) + (ad + bc)j	member
`/`	((ac + bd) + (bc − ad)j) / (c² + d²), error if c² + d² = 0	member
`==`	a = c and b = d (with a small tolerance for doubles)	member
`<<`	prints a + bj	friend

For symmetric forms like `2.0 * z`, make `+ - * /` friends instead. Magnitude is `sqrt(a² + b²)` through a `mag()` member.

Stage 3 – Implement and Verify (3 Marks)

```
#include <cmath>
#include <iostream>
#include <stdexcept>
class Complex {
   double re, im;
public:
   Complex() : re(0), im(0) {}
```

```
    Complex(double r) : re(r), im(0) {}
    Complex(double r, double i) : re(r), im(i) {}
    Complex operator+(const Complex& o) const { return {re + o.re, im + o.im}; }
    Complex operator-(const Complex& o) const { return {re - o.re, im - o.im}; }
    Complex operator*(const Complex& o) const {
        return {re * o.re - im * o.im, re * o.im + im * o.re};
    }
    Complex operator/(const Complex& o) const {
        double d = o.re * o.re + o.im * o.im;
        if (d == 0) throw std::domain_error("division by zero impedance");
        return {(re * o.re + im * o.im) / d, (im * o.re - re * o.im) / d};
    }
    bool operator==(const Complex& o) const {
        return std::fabs(re - o.re) < 1e-9 && std::fabs(im - o.im) < 1e-9;
    }
    double mag() const { return std::sqrt(re * re + im * im); }
    friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
        return os << z.re << (z.im < 0 ? " - " : " + ") << std::fabs(z.im) << "j";
    }
};
int main() {
    const double R = 10, L = 0.02, C = 50e-6;
    double w = 1000;
    Complex zR(R), zL(0, w * L), zC(0, -1.0 / (w * C));
    std::cout << "Series RLC: " << zR + zL + zC << '\n';
    w = 500;
    Complex zL2(0, w * L), zC2(0, -1.0 / (w * C));
    Complex lc = (zL2 * zC2) / (zL2 + zC2);          // L parallel C
    Complex par = (zR * lc) / (zR + lc);             // R parallel (L parallel C)
    std::cout << "Parallel RLC: " << par << "  |Z| = " << par.mag() << '\n';
    try { Complex bad = zR / Complex(0, 0); (void)bad; }
    catch (const std::exception& e) { std::cout << "Error: " << e.what() << '\n'; }
}
```

Program output (full test program, R = 10 Ω, L = 0.02 H, C = 50 μF):

```
w=1000: ZL = 0 + 20j, ZC = 0 - 20j
Series RLC: 10 + 0j  |Z| = 10
R || L: 8 + 4j
w=500: ZL = 0 + 10j, ZC = 0 - 40j
Series RLC: 10 - 30j  |Z| = 31.6228
Parallel RLC: 6.4 + 4.8j  |Z| = 8
(1+2j)+(3+4j) = 4 + 6j, |3+4j| = 5
(1+2j)*(3+4j) = -5 + 10j, (1+2j)/(3+4j) = 0.44 + 0.08j
Error: division by zero impedance
L||C at resonance (w=1000): Error: division by zero impedance
```

Hand verification:

- **ω = 1000:** ωL = 1000 × 0.02 = 20, so Z_L = 20j; 1/(ωC) = 1/0.05 = 20, so Z_C = −20j. Series: 10 + 20j − 20j = **10 + 0j**, |Z| = 10 (resonance). R ∥ L = (10 × 20j)/(10 + 20j) = 200j(10 − 20j)/500 = (4000 + 2000j)/500 = **8 + 4j**.
- **ω = 500:** Z_L = 10j, Z_C = − 1/(500 × 50e-6) = −40j. Series: 10 + 10j − 40j = **10 − 30j**, |Z| = √1000 = 31.62. L ∥ C = (10j × −40j)/(−30j) = 400/(−30j) = 13.33j. R ∥ (13.33j) = (10 × 13.33j)/(10 + 13.33j) = 133.33j(10 − 13.33j)/277.78 = **6.4 + 4.8j**, |Z| = √(40.96 + 23.04) = 8.
- **Complex arithmetic:** (1+2j)(3+4j) = 3 + 4j + 6j − 8 = −5 + 10j; (1+2j)/(3+4j) = (1+2j)(3−4j)/25 = (11 + 2j)/25 = 0.44 + 0.08j.

All program results match the hand calculations. **Division by zero:** dividing by 0 + 0j raises the `division by zero impedance` error, and so does L ‖ C at resonance (Z_L + Z_C = 0, an infinite parallel impedance).

Stage 4 – Defend the Design (2 Marks)

- **Versus `std::complex<double>`:** the standard class provides the same operators plus `abs`, `arg`, `polar`, `sqrt`, `exp` and is thoroughly tested and optimised. Dividing by zero there produces inf or NaN silently rather than raising an error.
- **Why each constructor and operator:** the default constructor lets arrays and uninitialised variables exist; the single-argument constructor creates resistors (and enables `Complex z = 5;`); the two-argument constructor creates inductors and capacitors. `+` combines series impedances, `*` and `/` combine parallel ones, `-` supports differences, `==` supports tests, and `<<` supports printing.
- **Remaining limitation:** `==` compares doubles with a fixed tolerance, which may not suit very large or very small values; there is also no `<` (complex numbers have no natural ordering), no conjugate or phase function, and the implicit conversion from `double` could hide mistakes.

Alternative: `std::complex<double>`

Every operation is O(1) time and uses two doubles of space in both versions. `std::complex` uses carefully written algorithms for division and magnitude (`std::abs` uses `hypot`, avoiding overflow), which makes it generally more accurate than the naive $\text{sqrt}(re^2 + im^2)$ shown here. Handling division by zero needs the same effort in both: explicitly check the divisor before dividing, because `std::complex` does not throw.